

# CS3.301: Operating Systems and Networks

## LECTURE 11&12: Virtualizing Memory: Address Spaces & the Memory API

---

INSTRUCTOR: DEEPAK GANGADHARAN

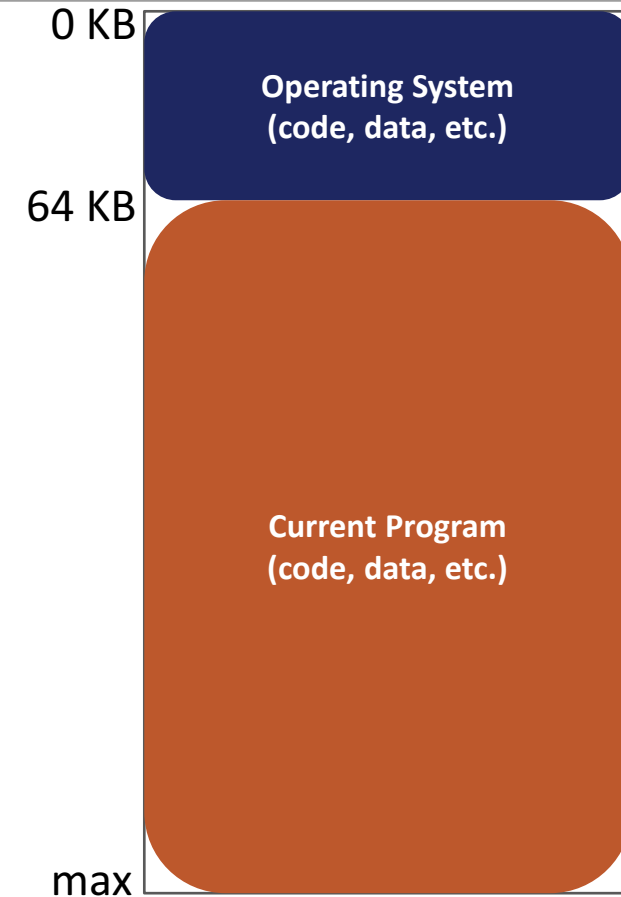
# Why Virtualize Memory?

---

- Every process needs to use memory — code, data, stack, heap
- But there is only ONE physical memory, shared by all processes and the OS
- We want each program to believe it has the whole memory to itself
- **This illusion is called virtualizing memory**

# Early Systems: No Abstraction At All

- In early machines, the OS was just a library sitting in memory
- One single running program used the rest of physical memory directly
- No illusions, no protection — but also very little was expected of the OS



Physical memory: one program at a time, no protection

# Multiprogramming and Time Sharing

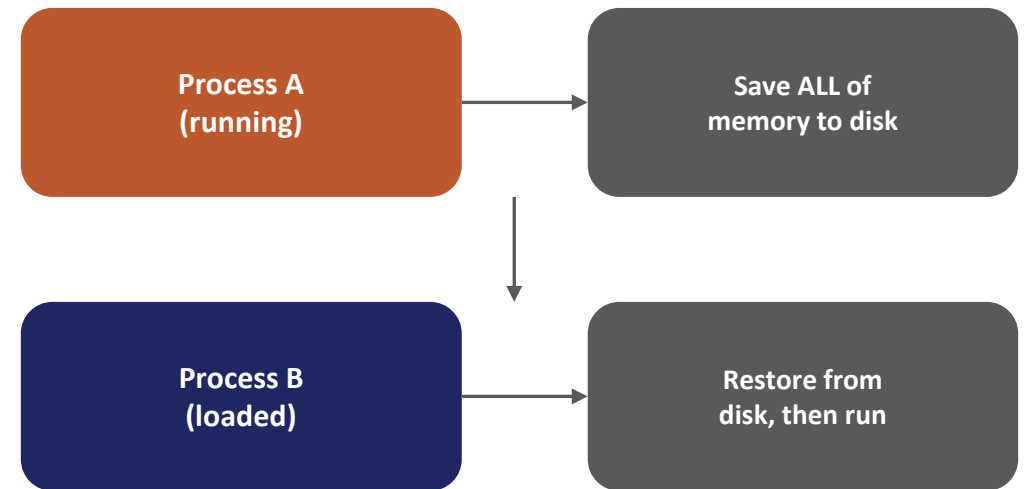
---

- Multiprogramming: keep several processes ready to run at once
- OS switches between them, e.g. when one performs I/O → better CPU utilization
- Time sharing (1959–1962): users want interactive, responsive systems
- Many users, each expecting a timely response from their task

# A Naive Way To Time-Share

---

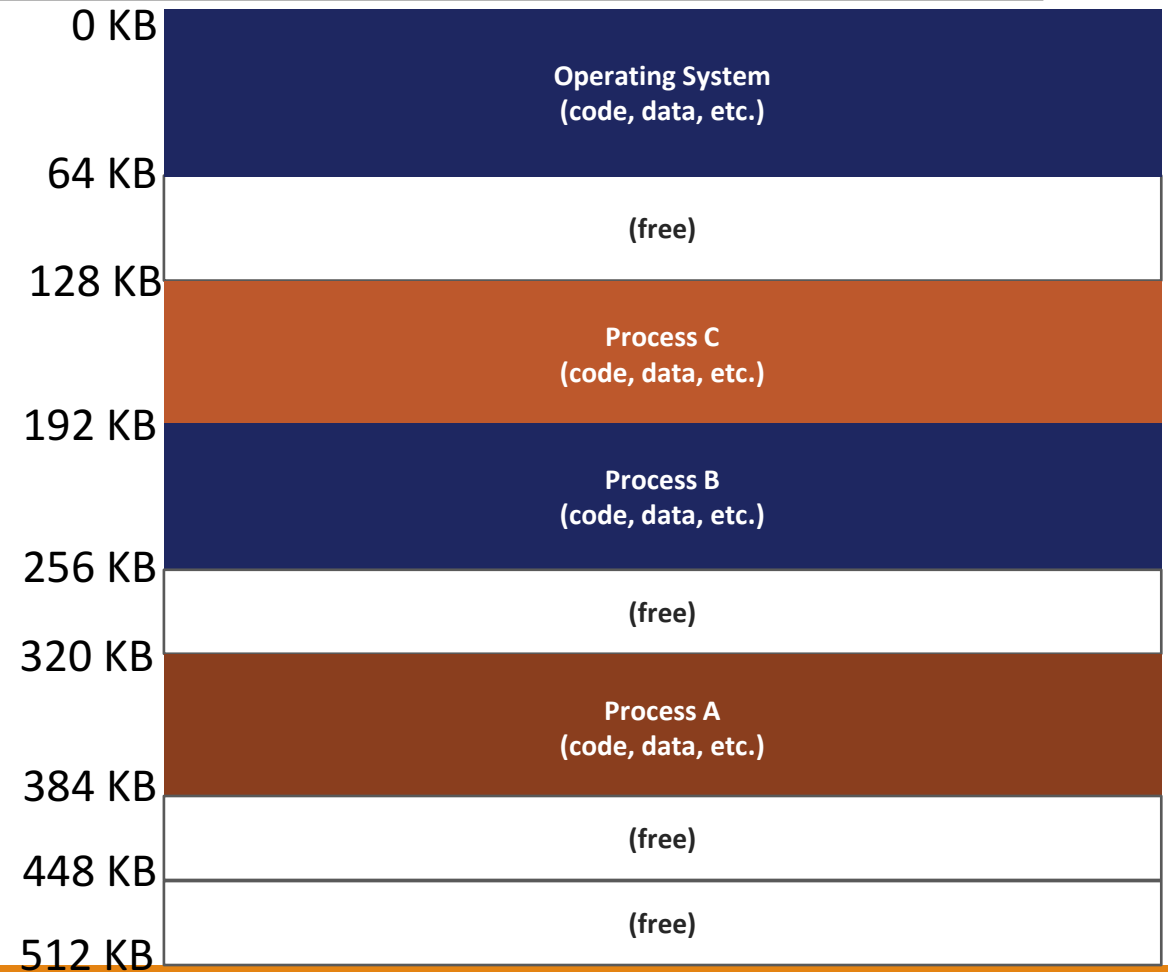
- Run process A fully in memory, then...
- Save ALL of memory to disk, load process B, run it
- Repeat: crude but workable sharing of the machine



**Problem: saving/restoring the FULL contents of memory to disk on every switch is far too slow, especially as memory grows.**

# A Better Idea: Leave Processes Resident

- Instead, leave processes sitting in memory while switching between them
- Each process gets a small carved-out slice of physical memory
- OS picks one to run (A); B and C wait in the ready queue



# A New Problem: Protection

---

- Multiple programs now reside in memory at the same time
- What stops process B from reading — or writing — process A's memory?
- **Protection becomes a first-class concern of the OS**
- We need an abstraction that gives each process its own private view

# The Address Space Abstraction

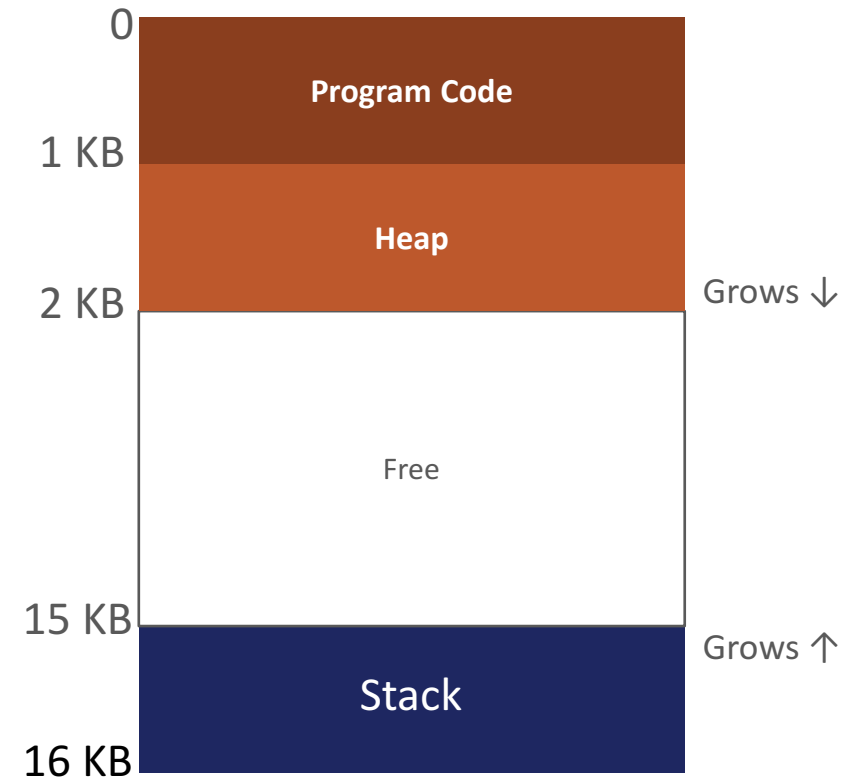
---

- **The address space is a process's view of memory in the system**
- Contains everything the running program needs:
  - Code — the instructions
  - Stack — function calls, local variables, return values
  - Heap — dynamically-allocated, user-managed memory (malloc/new)
- Understanding this abstraction is key to understanding virtual memory

# Example Address Space (16KB)

---

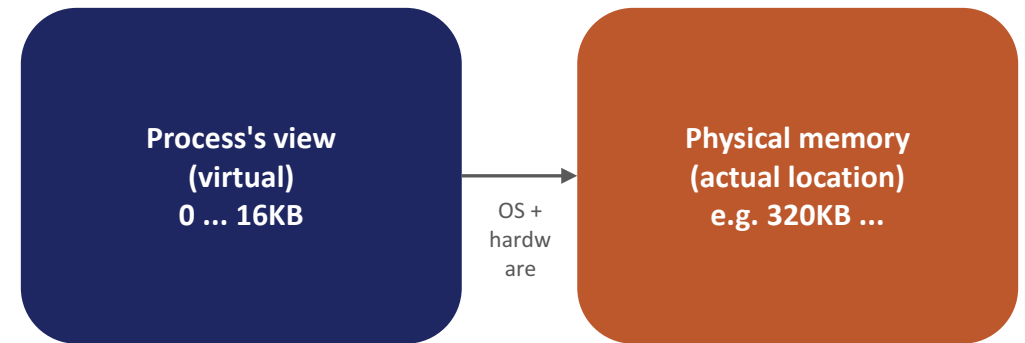
- Code is static — placed at the start of address space, fixed size
- Heap starts after code, grows downward → (e.g. malloc())
- Stack starts at the top, grows upward → (e.g. function calls)



# Every Address You See Is Virtual

---

- The address space is an illusion — the process is NOT really at address 0
- It is loaded at some arbitrary physical address, e.g. 320KB



A load at virtual address 0 is translated by the OS (with hardware support) to the real physical address where that process actually lives.

This translation is what it means to “virtualize memory.”

# The Crux: How To Virtualize Memory?

---

- **How can the OS build a private, potentially large address space for EVERY running process, on top of ONE single physical memory?**
- This is what it means for the OS to virtualize memory
- It requires serious hardware support (e.g. address-translation circuitry)

# Goal #1 — Transparency

---

- The virtualization should be invisible to the running program
- The program behaves as if it has its own private physical memory
- All the multiplexing work happens behind the scenes

## Transparency

invisible to the running program

## Efficiency

low time & space overhead (needs HW, e.g. TLBs)

## Protection

processes isolated from each other and the OS

## Goal #2 — Efficiency

---

- Virtualization should be efficient in time...
- ...and efficient in space (not too much memory for bookkeeping)
- Requires hardware support, e.g. TLBs (covered later in the course)



# Goal #3 — Protection

---

- Processes must be protected from one another...
- ...and the OS must be protected from processes
- A load, store, or instruction fetch must stay inside its own address space

## Transparency

invisible to the  
running program

## Efficiency

low time & space  
overhead (needs HW,  
e.g. TLBs)

## Protection

processes isolated  
from each other  
and the OS

# Isolation

---

- Isolation: if two entities are properly separated, one can fail
  - without affecting the other
- OS uses memory isolation to protect processes from each other...and to protect itself from the processes it runs
- Microkernels take this further — isolating pieces of the OS from each other

# The Memory API

---

- **Now we shift from the OS's view to the programmer's view**

How do UNIX/C programs actually allocate and manage memory?

What interfaces are used? What mistakes should be avoided?

A short, practical interlude before we go deeper into VM mechanisms

# Two Types of Memory: Stack vs. Heap

---

- Stack: automatic — compiler allocates & frees for you

Heap: explicit — you call malloc() and you call free()

**Stack memory  
(automatic)**

```
int x;
```

Managed implicitly by  
the compiler: allocated  
on call, freed on return.

**Heap memory  
(explicit)**

```
int *x = malloc(...);
```

Managed explicitly by  
the programmer: you  
malloc() it, you free() it.

# The malloc() Call

---

■ `#include <stdlib.h> void *malloc(size_t size);`

Takes a size in bytes, returns a pointer (or NULL on failure)

Best practice: use `sizeof()`, not a literal number

- `double *d = (double *) malloc(sizeof(double));`

The cast to `(double *)` reassures the reader, but isn't required for correctness

# sizeof()

---

■ `int *x = malloc(10 * sizeof(int));`

`printf("%d", sizeof(x));` → prints 4 or 8, NOT 40!

Here, `sizeof(x)` just measures the size of the POINTER

Compare: `int x[10];` `sizeof(x)` correctly gives 40

For strings: `malloc(strlen(s) + 1)` — +1 for the `\0` terminator

# The free() Call

---

■ `int *x = malloc(10 * sizeof(int)); ... free(x);`

Takes exactly one argument: a pointer returned earlier by `malloc()`

Notice: you do NOT pass the size — the allocator tracks that itself

Knowing WHEN and WHETHER to free is the hard part of memory management

# Common Memory Errors: Overview

---

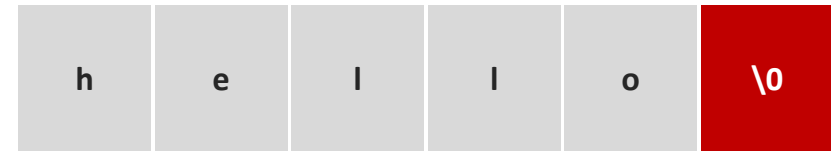
- Seven classic mistakes show up again and again - **All of these compile and often even RUN without complaint**

- 1 Forgetting to allocate memory
- 2 Not allocating enough memory (overflow)
- 3 Forgetting to initialize allocated memory
- 4 Forgetting to free memory (leak)
- 5 Freeing memory before you're done (dangling pointer)
- 6 Freeing memory repeatedly (double free)
- 7 Calling free() incorrectly (invalid free)

# Error Spotlight: Buffer Overflow

- `malloc(strlen(src))` forgets room for the `\0` terminator
- `strcpy()` then writes one byte past the allocated buffer
- May 'accidentally' work today — or corrupt memory / open a security hole

`malloc(strlen(src))` // too small!



allocated space | one byte past the end

`strcpy()` writes the string's terminating `\0` one byte past the buffer — a classic buffer overflow. It may run fine today and corrupt memory (or open a security hole) tomorrow.

# Error Spotlight: Leaks & Uninitialized Reads

---

- **Uninitialized read: using malloc'd memory before writing to it**

- Contents are garbage — could be zero (lucky) or anything else

- **Memory leak: forgetting to call free()**

- Harmless for short programs (OS reclaims all pages on exit)
- Fatal for long-running servers and especially inside the OS/kernel itself

# Error Spotlight: Dangling Pointers & Double Free

---

- **Dangling pointer: freeing memory before you're done using it**
  - Subsequent use can crash the program or corrupt other data
- **Double free: calling `free()` twice on the same pointer**
  - Result is undefined — the allocator's internal state can get corrupted
- **Invalid free: passing `free()` something `malloc()` never returned**

Tools like `gdb` and `valgrind` help find all of these

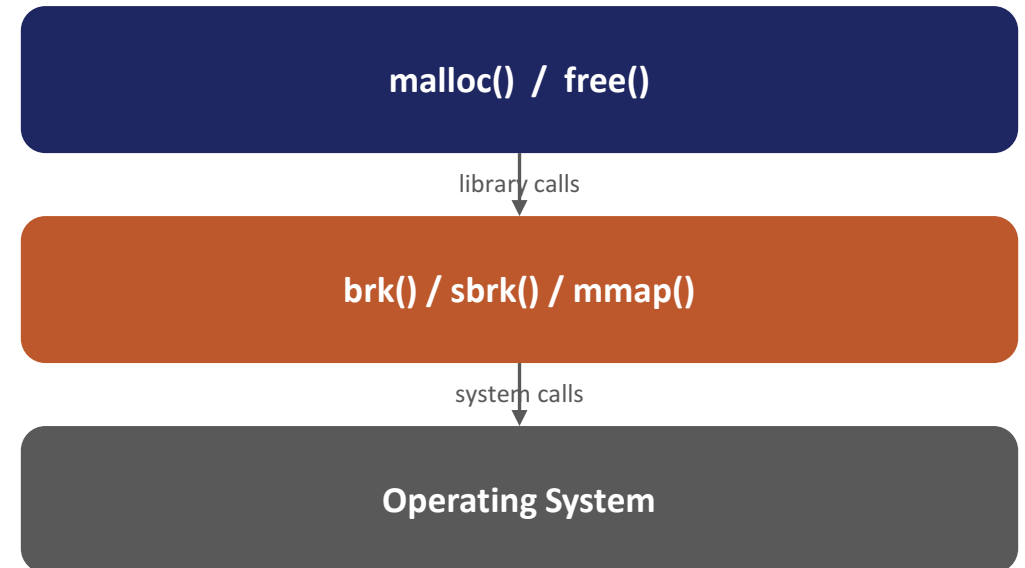
# Underlying OS Support & Other Calls

- `malloc()/free()` are library calls, not system calls

`calloc()`: allocates AND zeroes memory

`realloc()`: grows/shrinks an existing allocation, copying old data

**Key takeaway: allocate carefully, free exactly once, use your tools**



`brk()/sbrk()` move the program's break (end of heap); `mmap()` creates an anonymous memory region. Never call these directly — use `malloc()/free()`.

---

# Thank You!